

C# & .NET Fresher Cheatsheet

Quick-Reference & Interview Readiness Guide for Software Engineers

FRESHER / INTERN EDITION

1. AGENDA & TOPICS OVERVIEW

Page	Module	Core Concepts & Skills Covered
Page 1	Core Fundamentals	Data Types, Variables, Decision Making, Loops, Strings & Arrays
Page 2	Data Structures & Memory	Generic Collections, Stack vs. Heap, Value vs. Reference Types
Page 3	OOP Principles	Encapsulation, Inheritance, Polymorphism, Abstraction, Interfaces
Page 4	Interview Master Essentials	LINQ Queries, Exception Handling, Async/Await & Common Interview Q&As

2. VARIABLES, DATA TYPES & CONDITIONALS

Basic Data Types & Declarations

```
// Value types stored on Stack
int age = 22;
double salary = 45000.50;
bool isEmployed = true;
char grade = 'A';

// Reference type stored on Heap
string name = "John Doe";

// Implicit typing (Type inferred at compile time)
var rating = 4.8;
```

Decision Making (if-else & Switch)

```
int score = 85;
if (score >= 90) {
    Console.WriteLine("Grade A");
} else if (score >= 80) {
    Console.WriteLine("Grade B");
} else {
    Console.WriteLine("Grade C");
}

// Modern Switch Expression (C# 8+)
string result = score switch {
    >= 90 => "Excellent",
    >= 75 => "Good",
    _ => "Needs Improvement"
};
```

3. CONTROL LOOPS & ITERATION

For, While & Do-While Loops

```
// Standard For Loop
for (int i = 0; i < 3; i++) {
    Console.WriteLine($"Count: {i}");
}

// While Loop
int count = 0;
while (count < 2) {
    count++;
}
```

Foreach Loop (Arrays & Collections)

```
string[] techStack = { "C#", ".NET", "SQL" };

foreach (string tech in techStack) {
    Console.WriteLine($"Tech: {tech}");
}
```

4. C# CORE CONTAINERS & DATA STRUCTURES

Choosing the correct data structure is a frequent interview screening topic for entry-level developers.

List<T> (Dynamic Array)

```
List<string> fruits = new List<string>
();
fruits.Add("Apple");
fruits.Add("Banana");
fruits.Remove("Apple");

Console.WriteLine(fruits[0]); // Output:
Banana
```

Dictionary<K, V> (Key-Value Lookup)

```
Dictionary<int, string> empMap = new();
empMap.Add(101, "Alice");
empMap[102] = "Bob";

if (empMap.ContainsKey(101)) {
    Console.WriteLine(empMap[101]);
}
```

Queue<T> (First-In, First-Out)

```
Queue<string> queue = new();
queue.Enqueue("Task 1");
queue.Enqueue("Task 2");

string current = queue.Dequeue(); //
Task 1
```

Stack<T> (Last-In, First-Out)

```
Stack<string> history = new();
history.Push("Page A");
history.Push("Page B");

string top = history.Pop(); // Page B
```

5. MEMORY ARCHITECTURE: STACK VS. HEAP

Feature	Stack Memory	Heap Memory
Type Stored	Value Types (`int`, `float`, `bool`, `struct`)	Reference Types (`class`, `string`, `array`, `object`)
Allocation	Static execution allocation (Fast)	Dynamic memory allocation (Managed)
Deallocation	Automatic when scope/method ends	Cleaned up by .NET Garbage Collector (GC)

Interview Tip: Value vs. Reference Types

When you pass a **Value Type** into a method, a copy of the data is created. When you pass a **Reference Type**, a pointer reference to the underlying memory address on the Heap is passed.

Value vs. Reference Demonstration

```
// Value Type Example
int val1 = 10;
int val2 = val1;
val2 = 20; // val1 remains 10

// Reference Type Example
int[] arr1 = new int[] { 1, 2 };
int[] arr2 = arr1;
arr2[0] = 99; // arr1[0] is NOW ALSO 99 because both point to same memory!
```

6. OBJECT-ORIENTED PROGRAMMING (OOP) IN C#

Mastering the 4 core pillars of OOP with functional C# examples is essential for cracking technical interviews.

1. Encapsulation

Hiding private internal state and exposing access through public properties/methods.

```
public class BankAccount {  
    private double _balance; // Private field hidden from outside  
  
    public double Balance => _balance; // Read-only getter  
  
    public void Deposit(double amount) {  
        if (amount > 0) _balance += amount;  
    }  
}
```

2. Inheritance

Reusing code from a base (parent) class in derived (child) classes.

```
public class Animal {  
    public string Name { get; set; }  
    public void Eat() => Console.WriteLine("Eating...");  
}  
  
public class Dog : Animal { // Dog inherits Name and Eat()  
    public void Bark() => Console.WriteLine("Woof!");  
}
```

3. Polymorphism (Method Overloading & Overriding)

Ability of objects to take on different behaviors dynamically or at compile time.

```
public class Calculator {  
    // Compile-Time Polymorphism (Method Overloading)  
    public int Add(int a, int b) => a + b;  
    public double Add(double a, double b) => a + b;  
}  
  
public class Shape {  
    // Run-Time Polymorphism (Method Overriding)  
    public virtual void Draw() => Console.WriteLine("Drawing Shape");  
}  
  
public class Circle : Shape {  
    public override void Draw() => Console.WriteLine("Drawing Circle");  
}
```

4. Abstraction & Interfaces

Hiding complex implementation details using interface or abstract class.

```
public interface IPaymentProcessor {  
    void ProcessPayment(double amount); // Contract only  
}  
  
public class PayPalProcessor : IPaymentProcessor {  
    public void ProcessPayment(double amount) {  
        Console.WriteLine($"Paid {amount} via PayPal");  
    }  
}
```

7. LINQ (LANGUAGE INTEGRATED QUERY) ESSENTIALS

LINQ allows querying collections directly in C# using readable, SQL-like methods.

```
List<int> numbers = new() { 1, 2, 3, 4, 5, 6 };

// Filter even numbers (Where)
var evens = numbers.Where(n => n % 2 == 0).ToList(); // [2, 4, 6]

// Transform elements (Select)
var squares = numbers.Select(n => n * n); // [1, 4, 9, 16, 25, 36]

// Ordering and First item
int firstBig = numbers.OrderByDescending(n => n).FirstOrDefault(); // 6
```

8. EXCEPTION HANDLING & ASYNC PROGRAMMING

Try-Catch-Finally Block

```
try {
    int x = 10;
    int result = x / 0;
} catch (DivideByZeroException ex) {
    Console.WriteLine("Cannot divide by
zero!");
} finally {
    // Runs regardless of error
    Console.WriteLine("Cleanup
completed.");
}
```

Async / Await (Non-blocking)

```
public async Task<string>
FetchDataAsync() {
    HttpClient client = new();
    string data = await client
.GetStringAsync("https://api.com");
    return data;
}
```

9. HIGH-FREQUENCY FRESHER INTERVIEW Q&A

Q1: What is the difference between Array and List in C#?

Array: Fixed size allocated at initialization. Faster access, stored sequentially in memory.

List<T>: Dynamic size, resizes automatically as items are added.

Q2: What is the difference between Abstract Class and Interface?

An **Abstract Class** can contain implementations of methods, fields, and constructors, but supports single inheritance. An **Interface** defines contracts only (until C# 8), supports multiple inheritance, and contains no state/fields.

Q3: What is Boxing and Unboxing?

Boxing: Converting a Value Type to a Reference Type (`object obj = 42;`).

Unboxing: Extracting the Value Type back from the object reference (`int x = (int)obj;`).